INDEPENDENT RESEARCH
Computer Science · Artificial Intelligence

TECHNICAL REPORT

Kiodai: Structured Prospective Memory for Language-Model AI Agents and Its Evaluation Methodology

Prospective memory in language-model agents
Experimental methodology and evaluation

Bagdat Beimzhan
Independent Researcher

Astana, Kazakhstan · 2026

CONTENTS

Abstract....3
1. Introduction....3
2. Theoretical Background and Related Work....4
2.1. From Remembering to Executing an Intention....4
2.2. Related Memory Evaluations and Structured Stores....4
3. Systems and Implementation....4
3.1. Historical and Final Conditions....4
3.2. From Instruction to Execution Receipt....5
4. Experimental Methodology....6
4.1. Scenarios, Freeze Procedure, and Run Conditions....6
4.2. Evaluator, Failures, and Unit of Analysis....7
5. Results....8
5.1. Pilots and Functionality Check....8
5.2. Main Comparison....8
5.3. Resource Use and Obligation Accounting....9
6. Failure Analysis and Discussion....11
6.1. Observed Failure Stages....11
6.2. Execution Chains and a Possible Explanation....11
7. Limitations, Practical Relevance, and Author Contribution....12
8. Conclusion....13
9. References....14

Abstract

This study evaluates prospective memory in language-model agents: executing earlier intentions at the appropriate moment. The initial hypothesis predicted benefits from a fixed Kiodai instruction; the investigation later expanded to explicit intention storage and control mechanisms. The A0/A1 pilots showed no improvement. A0, B_ledger, and A2 using DeepSeek V3.1 were then compared on 12 trajectories from four template families, with one run per method-trajectory pair. The scenarios were exposed during development. The primary measure, mean trajectory Set-F1, was 0.8643, 0.8111, and 0.4806, respectively. The primary A2 − B_ledger difference was −0.330556. The improvement hypothesis was not supported under these conditions. The contribution comprises an implemented system, a frozen comparison, and record-based failure analysis. Intention stores are prior work. The author's contribution covers the research question and comparison methodology; AI assisted coding, writing, and semantic review, without an independent human annotator. Potential applications include evaluating educational and organizational assistants; practical benefits were not tested.

1. Introduction

A user might instruct an assistant: “When the cooling board becomes stable, archive the report.” Other messages arrive between the instruction and the appropriate moment. The condition may change, the instruction may be cancelled, or execution may depend on completing another task. The agent must retain the current instruction, obtain permitted evidence when necessary, select an action, and verify its execution outcome. Reproducing the original wording alone is insufficient.

Prospective memory means carrying out an earlier intention when the relevant time or event occurs. Retrospective memory answers a question about the past, such as which deadline was specified. Prospective memory requires applying that knowledge at the appropriate moment without another explicit reminder. PM-Bench studies this behaviour in a text-based simulator with ongoing activities, delayed actions, and queries about the state of the environment [1].

The practical relevance concerns assistants that manage deadlines, revisions, and conditional reminders. A useful system must execute required actions while avoiding premature or cancelled ones. Educational and organisational situations motivate this project. The study does not measure academic attainment, application usability, or reliability in real-world deployment.

The object of study is the behaviour of a language-model agent over a sequence of interactions. The specific focus is how instructions, explicit intention representations, and monitoring policies affect timely action selection. The aim is to implement Kiodai and evaluate it through a controlled comparison while retaining evidence of failures and the limits of the findings.

The research developed in two stages. The first tested whether a fixed instruction, P1, improved the same agent by comparing baseline A0 with A1. After pilots showed no observed improvement, a separate architectural stage was introduced. It compared A0, the structured B_ledger variant with model-directed queries, and A2 with bounded programmatic monitoring. This extension was not the original A0/A1 plan; it was documented in a separate protocol.

The research questions are whether the added instruction helps in the pilots; whether A2 produces a positive mean change in Set-F1 relative to B_ledger; and how performance relates to resource use and observed failures. Improvement was initially expected from the instruction and later from the architectural extension. The methods comprise a literature review, implementation

of a simulator-based agent, local checks, paired comparison, and analysis of saved logs. Conclusions are restricted to the implemented system, one model configuration, and the scenarios used.

2. Theoretical Background and Related Work

2.1. From Remembering to Executing an Intention

In the Virtual Week paradigm, delayed tasks are embedded in simulated everyday activities. Rendell and Craik examine prospective remembering in a laboratory board game and in everyday life [2]. The relevant principle for Kiodai is the combination of ongoing activity with future obligations. An experiment involving a language-model agent is not a study of human memory and does not justify transferring psychological explanations to AI.

PM-Bench adapts this setting to a text-based agent. The agent receives a history, a scene, an action menu, and available state channels. Its published conditions already include a todo-ledger: a contextual list of intentions, conditions, and statuses [1]. Kiodai uses the existing simulator and evaluator, but its final comparison uses separately constructed development scenarios. It is not a new result on the released final week of PM-Bench.

2.2. Related Memory Evaluations and Structured Stores

TriggerBench tests timely intervention based on previously stated constraints and compares it with retrospective questions. Positive, negative, and additional-task variants help distinguish useful action from a tendency to issue reminders indiscriminately [3]. This motivates attention to false actions and cues that resemble, but do not satisfy, the intended condition. TriggerBench's metrics and conditions differ from the Set-F1 evaluation used here.

MemoryAgentBench, presented by Hu, Wang, and McAuley, evaluates accurate retrieval, learning during interaction, long-context understanding, and selective forgetting [4]. It therefore addresses a broader memory-evaluation problem. Kiodai examines a particular intention-execution chain and does not claim to cover all four capabilities.

The Prospective Intention Store, PIS, places intentions in a typed external store. Its description separates formation, revision, filtering, and action selection: the model performs bounded language operations, while programmatic rules manage parts of the lifecycle [5]. This work is a direct predecessor of Kiodai's architectural stage. Kiodai reconstructs selected PIS ideas from the paper; faithful reproduction of PIS and comparability of all its settings have not been established.

Intention stores, version control, scheduling, and separation between agent and evaluator are not claimed as inventions of this project. The concrete contribution is an implementation, a frozen comparison of two monitoring policies with a shared store, and traceable failure analysis. Scores from other publications are not treated as results of the same experiment: the models, scenarios, computational resources, and execution rules differ.

3. Systems and Implementation

3.1. Historical and Final Conditions

A0 uses a baseline instruction, its own interaction history, and permitted tools. A1, also labelled P1 in historical records, adds a fixed prospective-memory instruction to the same agent.

These pilots did not add an external store. A1 is not one of the three conditions in the final comparison.

In B_ledger and A2, the model extracts intentions into structured records. The verified code shares Agent, Store, Gateway, the extraction schema, and prompt texts between these methods. Both support versions, dependencies, cancellations, binding validation, and status updates from execution receipts. Their main difference concerns the choice of state queries: B_ledger delegates it to the model, whereas A2 uses a programmatic policy [6].

Table 1. Experimental conditions and shared functionality.

Feature	A0	A1 / P1	B_ledger	A2
Evaluation stage	Pilots and final	Pilots only	Final	Final
History and tools	Available	Available	Available	Available
P1 addition	No	Yes	Different protocol	Different protocol
Intention store	No	No	Yes	Yes
Stored versions, dependencies, cancellations	No store layer	No store layer	Shared Store	Shared Store
Receipt-based store updates	No store	No store	Yes	Yes
Query decision in final comparison	Model	Not included	Model	Controller

The absence of a separate store in A0 does not imply an absence of information about obligations. In the final comparison, all methods receive the available history, current menu, and reports of their own action outcomes. Their histories naturally diverge after different queries and actions. A0 can make an additional model call after a query, so it is not restricted to one model call per checkpoint.

3.2. From Instruction to Execution Receipt

The earlier notation I = (c, a, s, v), where c denotes a condition, a an action, s a state, and v a version, served as an analytical specification. The later implementation stores a richer record: its own identifier, condition type and text, day and time when available, channel, dependencies, quotations from available observations, status, version, and execution-attempt information. Data and events are persisted in SQLite. Formally complete fields do not guarantee semantic correctness [6].

At a checkpoint, the model proposes operations to create or revise records. The program validates structure, references to observations, and permitted operations. An incompletely specified intention may be quarantined, temporarily excluding it from execution. After successful extraction, the A2 controller considers channels associated with eligible unfinished intentions and prioritises the channel that has gone longest without a check. At most one query is permitted per checkpoint. If the time is already visible in the scene, a time-based intention does not require an additional clock query.

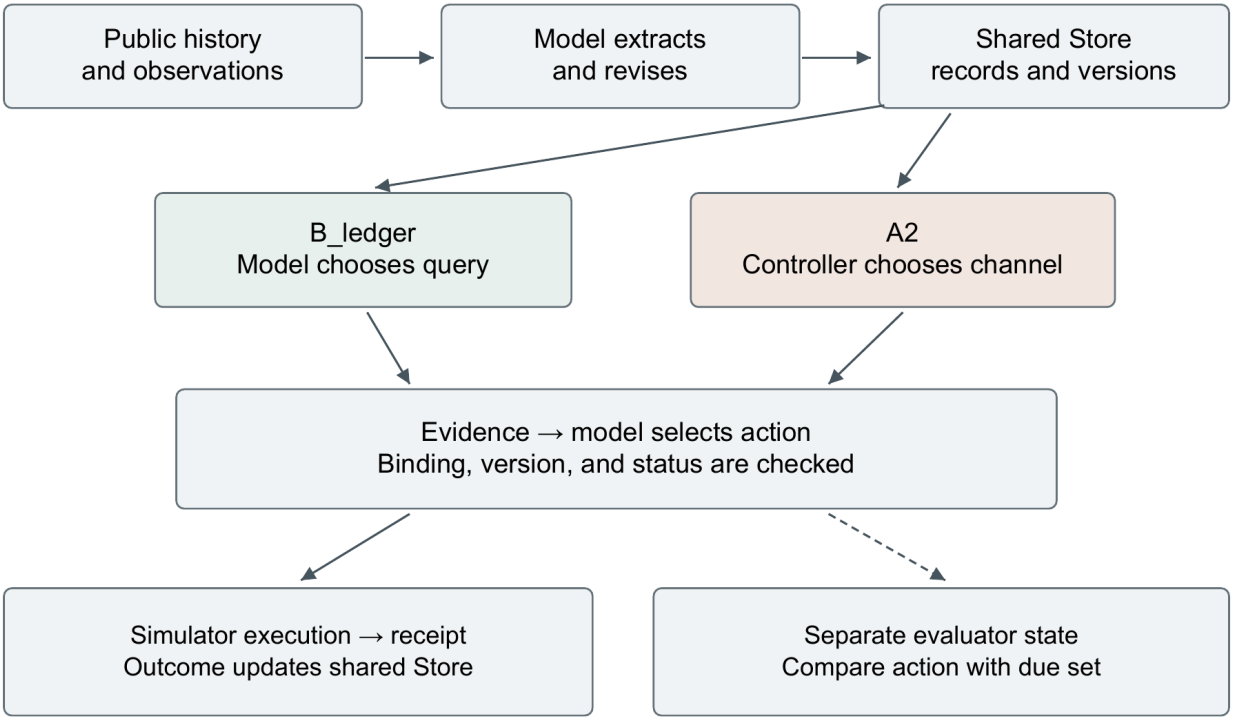


Figure 1. Intention processing in B_ledger and A2. The shared stages are identical; the highlighted branches show the difference in query policy. The evaluator uses separate state after actions have been selected.

The model then matches the current observation to the condition and binds a selected action identifier from the menu to an intention and its version. Programmatic eligibility checks validate status and dependencies, but do not establish correct semantic alignment with the action, day, or event. A stored value of “DayTwo, 11:00” does not itself guarantee that an action on DayOne will be prohibited.

An attempt is recorded after selection. An execution receipt reports the simulator's outcome. A successful receipt moves the stored intention to completed; unsuccessful and uncertain outcomes are distinguished. This separates selection from completion, but an incorrect binding can associate the outcome with the wrong record. Cancellation rules and rejection of an old version number also do not prevent an old condition from being reintroduced in a new version. Exactly-once execution of real external actions was not tested.

4. Experimental Methodology

4.1. Scenarios, Freeze Procedure, and Run Conditions

The final v2.1-comparison-v1 protocol contains 12 synthetic trajectories: three variants in each of four families. The hidden family requires querying hidden state; visible_events uses visible cues and their revisions; cross_day moves an obligation to a different day; and revision includes rescheduling and cancellations. Each trajectory has eight checkpoints, distributed over two days in cross_day. All scenarios were exposed during development and MOCK checks; hidden_91320 overlaps with earlier smoke tests [6].

A0, B_ledger, and A2 were each run once on every trajectory, giving 36 runs and 288 checkpoints. There were no additional repetitions, exclusions, or selective reruns. Family and variant order is determined by seed 20260910; method order rotates cyclically and is balanced across positions. Checkpoints are scheduled by the simulator. Heartbeat is disabled: A2 receives

no additional opportunities between checkpoints and does not demonstrate autonomous background operation.

The candidate implementation was frozen at commit 959db38dac8b68f63ecf79420dcd53bea2278cf2; the experiment on 11 September 2026 ran from cd0ce89e10d036918d1af06e5f5f2140930a0981. Experiment-support code has a separate version, 832d6ff. The manifest records hashes for 59 source, configuration, and scenario files. The archive retains 445 original run files. Their hashes and official scores were rechecked during manuscript preparation [6, 7].

Table 2. Frozen settings for the final comparison.

Parameter	Value
Model and API identifier	DeepSeek V3.1; deepseek/deepseek-chat-v3.1
Route	OpenRouter / Novita; route fallbacks disabled
Generation	temperature = 0; top_p = 1; seed = 20260904
Additional request settings	fp8; reasoning.enabled = false; reasoning.exclude = true; require_parameters = true
Maximum output	A0: 256 tokens; extraction: 3072; B_ledger/A2 selection: 1536
Context and tools	Full own history; at most 1 environment query per checkpoint; heartbeat = false
Failures and limits	1 validation retry; 0 transport retries; 120 s timeout; request size at most 48,000 bytes

Saved requests confirm the model, generation parameters, limits, and route; responses report DeepSeek V3.1 and Novita. These records do not independently verify how the provider implemented every parameter internally. Actual computation is unequal: structured methods use a separate extraction stage and longer outputs. Retries after failures also contribute to resource use.

4.2. Evaluator, Failures, and Unit of Analysis

The agent receives a public frame containing history, observations, the menu, available channels, and the results of its own actions. Canonical expected identifiers and due sets remain with the environment and evaluator. A separate runner is used instead of the original PM-Bench path that substitutes the due set after an unparseable response. If extraction or selection remains invalid after the retry, the system performs no target task action and records a failure. Choosing A to continue the current scene is not completion of a deferred obligation [6].

Inspection of the data boundary and saved requests supports separation in this run. It is not a universal proof that every possible leakage channel is absent. Logs retain raw requests and responses, retries, actions before and after identifier mapping, scores, tokens, call durations, and costs [7].

The due set is computed from evaluator state: current tasks, time, events, cancellations, and completions that actually occurred. It is not the list of tasks that the agent believes are ready. For a hidden cue, the evaluator has access to the environmental state whether or not the agent queried it. An unfinished prerequisite can prevent a dependent task from becoming due. The number of obligations that ever require execution can therefore differ between methods even when their initial scenario is identical.

Within a trajectory, TP counts correct selections at the relevant checkpoints, FP counts extra selections, and FN counts omitted actions in the due set. The unchanged Set-F1 rules are used. Set-F1 measures agreement between action sets, rather than an ordinary proportion of correct answers [1, 7].

$$Precision = TP/(TP+FP)$$

$$Recall = TP/(TP+FN)$$

$$Set-F1 = 2TP/(2TP+FP+FN)$$

Set-F1 is first computed separately for each trajectory. The primary measure for a method is the arithmetic mean of its 12 trajectory values, giving each trajectory equal weight. The primary contrast is the mean of the 12 paired A2 − B_ledger differences; the secondary A2 − A0 contrast is calculated separately. Micro Set-F1 is calculated differently: TP, FP, and FN are pooled across trajectories before applying the formula. Trajectories with different denominators consequently have different influence on the micro score.

If a denominator is zero, the auxiliary calculation returns an undefined value rather than assigning a score. All Set-F1 denominators are positive in these 36 runs. No confidence intervals, standard deviations, or significance tests are claimed. Model calls and checkpoints are not independent replications, and the 12 trajectories share four templates.

5. Results

5.1. Pilots and Functionality Check

In the initial pilot, A0 and A1 both achieved Set-F1 = 1.00: TP = 5, FP = 0, and FN = 0. In the longer, three-day follow-up, both conditions had TP = 11, FP = 0, and FN = 1, giving Set-F1 = 22/23 ≈ 0.9565. A1 used more input tokens. These two comparisons showed no improvement; the first pilot had a ceiling that left no room for a higher score. The findings do not establish that prompting is generally ineffective [8].

Architectural development included a failed v2 check with an empty store, followed by a repaired v2.1 implementation. In a separate live v2.1 smoke test, eight checkpoints and three intentions were completed: TP = 3, FP = 0, FN = 0, and Set-F1 = 1.00, with seven monitoring queries and three successful receipts. This was a functionality check on a known scenario without a comparison condition. Even this run contained field errors followed by corrections [9].

5.2. Main Comparison

Table 3 reports the final scores. Mean and micro values differ by definition, not because of rounding. A0 has the highest mean trajectory Set-F1. A2 underperforms both simpler conditions [7].

Table 3. Performance in the frozen comparison, with 12 trajectories per method.

Method	TP	FP	FN	Micro Set-F1	Mean trajectory Set-F1
A0	22	6	2	0.8462	0.8643
B_ledger	18	4	3	0.8372	0.8111
A2	12	5	7	0.6667	0.4806

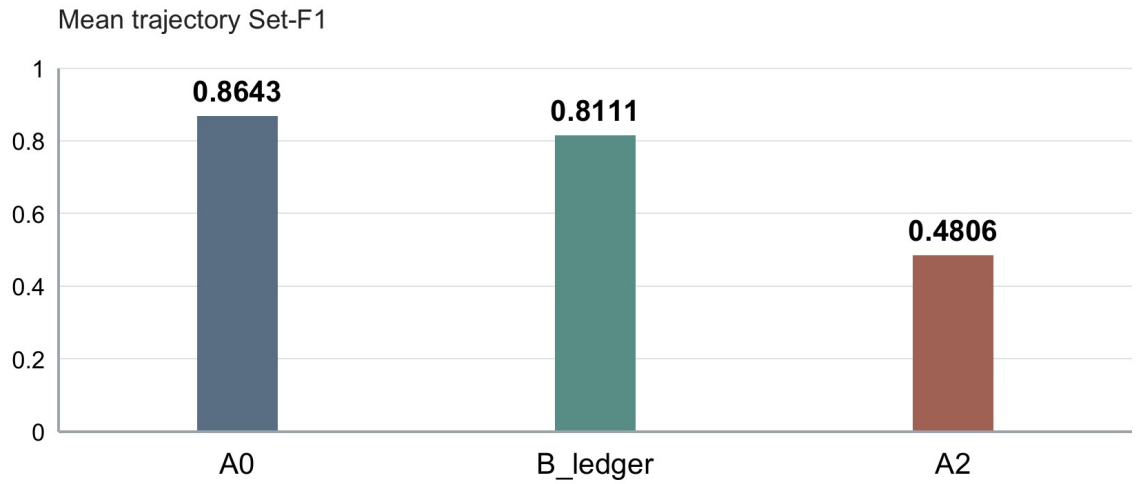


Figure 2. Primary measure: mean Set-F1 across 12 trajectories. The axis spans 0 to 1 and bars start at zero. Values were recalculated from saved scores; uncertainty was not estimated.

The primary paired difference, A2 − B_ledger, is −0.330556 F1 units; the secondary difference, A2 − A0, is −0.383730. These six-decimal values were reconstructed from trajectory-level data, not by subtracting the rounded four-decimal table entries. The improvement hypothesis was not supported in this comparison. This is a descriptive conclusion, without a claim of statistical rejection.

Results vary across families: A2's mean is 0.0000 in cross_day and 0.8667 in visible_events, compared with 0.8000 for B_ledger in the latter family. The small visible_events advantage occurred without tool queries and does not establish a benefit from monitoring. On v2_hidden_91320, the scenario overlapping with the smoke test, A2 obtained Set-F1 = 0 despite a non-empty store and six queries. The earlier successful smoke test did not guarantee repeated success. Historical runs are not pooled with the final comparison as additional replications [7, 9].

5.3. Resource Use and Obligation Accounting

Table 4. Actual model usage and API-reported costs.

Method	Calls	Input tokens	Output tokens	Cost, USD
A0	117	177,855	3,175	0.04047
B_ledger	251	910,740	44,534	0.21212
A2	234	843,540	42,889	0.19871
Total	602	1,932,135	90,598	0.45130

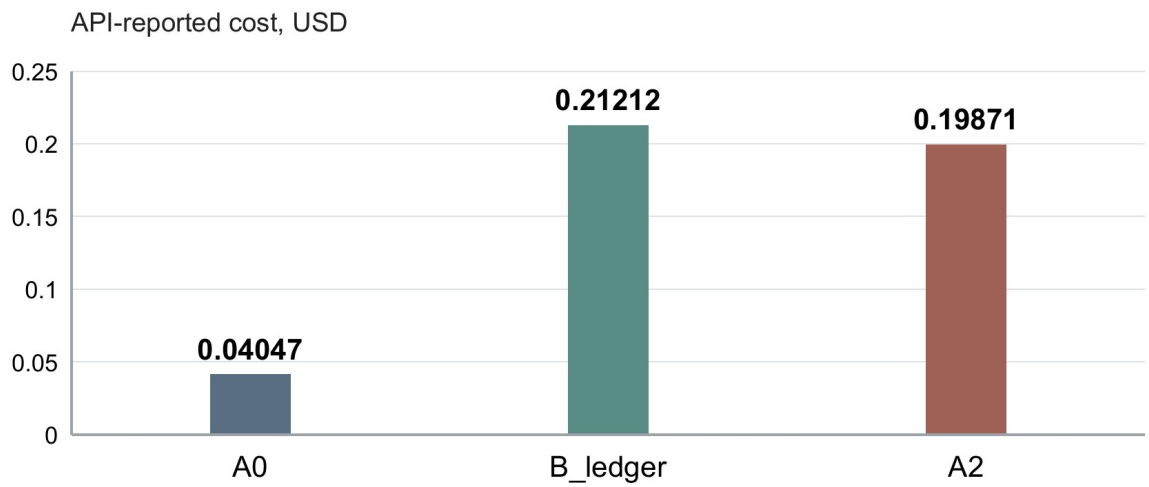


Figure 3. API-reported cost of this experiment in USD. Bars start at zero. These values describe the recorded run, not current prices or the full development budget.

Unrounded records give costs of USD 0.19870816 for A2 and USD 0.04047361 for A0. Their ratio is 4.90957, approximately 4.91. A0 combines the highest primary score with the lowest reported cost. The total reported by the API is USD 0.45130261. No independent reconciliation with the provider's final invoice is available. Costs include extraction, selection, and retries after errors. Token counts and costs are not used to infer latency [7].

Table 5. Queries, actions, and receipts in the final comparison.

Method	Hidden-board queries	Query-supported successes	Executions	Successful receipts	Selection-validation failures
A0	15	6	28	22	2
B_ledger	8	4	22	18	24
A2	18	3	17	12	15

A query-supported success means that a response from the relevant channel is available at the same checkpoint and is consistent with a successfully selected action requiring hidden-state evidence. This is an observable basis for the action, not proof of the query's internal causal role. Several actions may rely on one response. Selection-validation failures are counted by attempt, including retries. They must not be divided by all model calls: those calls also include extraction, and the summary table does not provide the selection-attempt denominator.

Table 6. Original obligations accounted for using evaluator state.

Method	Instructed	Ever due	Dependency-blocked	Completed	Unfinished, excluding cancelled	Cancelled
A0	27	24	0	22	2	3
B_ledger	27	21	3	18	6	3
A2	27	19	5	12	12	3

Three cancellations in the revision family explain the initially unclear remainder. For every method, completed + unfinished excluding cancelled + cancelled = 27. A0's blocked count, absent from the short summary, was recalculated from the logs as zero. This is a measured value, not a substitution for missing information. Blocking is recorded when a trigger occurs while a prerequisite remains incomplete [7].

For A2, five dependent obligations never became due: two in hidden and three in cross_day. Its 12 unfinished obligations therefore differ from its seven official FN. A reduction in due obligations caused by earlier failures is not an improvement. The obligation table supplements the unchanged metric and makes these consequences visible.

6. Failure Analysis and Discussion

6.1. Observed Failure Stages

The post-run semantic review covered 24 cases from the structured methods; the 12 A0 behavioural trajectories were reviewed separately. The review was AI-assisted, without an independent human annotator. Key examples were checked against raw responses, store snapshots, and receipts for this paper. Table 7 shows symptoms and the earliest evidenced stages, without assigning category frequencies or treating the categories as mutually exclusive [7, 10].

Table 7. Failures in the frozen comparison and limits of interpretation.

Symptom	Earliest evidenced stage	Consequence	Alternative explanation or limit	Evidence
No record by the deadline	Applying extraction: batch rejected for a dependency reference	Registration missed	Drafts name the actions; this does not prove a lack of understanding	A2 hidden_91320, checkpoints 1–3
Dependency omitted	Accepted extraction without the dependency	Eligibility changes; a later task may remain blocked	The omission's cause and the prompt's role are not isolated	A2 hidden_91320, checkpoints 3–4
Old deadline returns	Accepted new record version	Premature action and unsuccessful receipt	Revision and selection errors coexist	A2 revision_91300, checkpoints 4–7
Correct future day stored; action today	Selection and binding to the wrong menu item	False action; required registration remains incomplete	Correct storage does not remove selection error	A2 cross_day_91331, checkpoint 3
Positive board response without execution	Selection validation after the query	Empty set of target actions	Too few queries do not explain this checkpoint	A2 hidden_91320, checkpoint 7
Superseded event cue returns	Accepted condition revision	Premature sealing	A later correction may conceal the defect in a high F1 score	A2 visible_events_ 91310; B_ledger, visible_events family

Case labels in the table are shortened: every scenario name has the prefix v2_. For the first five rows, raw logs are available and were checked against the descriptions. The last row relies on a saved AI annotation with pointers to responses. Full paths and line numbers are recorded in the existing verification materials. Earlier v2 bugs involving a mismatched extraction contract are excluded from this table. They were repaired before the final run, so they cannot explain the entire final deficit without additional evidence.

6.2. Execution Chains and a Possible Explanation

For A2 on v2_hidden_91320, extraction batches were rejected at the first two checkpoints, leaving the store empty. Intentions appeared at checkpoint 3, but the registration deadline had

already passed. The sealing dependency was initially absent and was later restored. At checkpoint 7, a query to board_cobalt91320 returned positive evidence; selection and its permitted retry failed validation, producing no target actions or receipts. The hidden evidence was obtained, but the chain did not reach completion [7, 10].

On v2_revision_91300, A2 reintroduced the 12:00 deadline after it had been moved to 13:00. At checkpoint 6 it selected an action prematurely and received simulator_not_completed; at checkpoint 7 it restored 13:00 and received simulator_completed. All 16 model responses in this case passed validation. Correct formatting and valid references therefore did not guarantee that the revision preserved the intended meaning [7, 10].

On v2_cross_day_91331, the stored record specifies DayTwo, yet at checkpoint 3 on DayOne the agent selects a distractor menu item for a spare sample and binds it to the real registration intention. The receipt is unsuccessful, and registration is not completed later. This shows a separate selection and binding failure despite a correct day field. The case does not support attributing every problem to extraction [7, 10].

A working chain is also present. For A2 on v2_hidden_91322, registration completed after a dependency correction and a selection retry. At checkpoint 7, positive board evidence was used to select sealing and archiving; both actions received successful receipts. A final Set-F1 of 1.00 coexists with intermediate errors. A0 also completed three tasks on the same scenario, using fewer queries and model calls. This example demonstrates that the chain can work, but does not establish superiority [7, 10].

Taken together, the observations are consistent with possible semantic error propagation through intermediate representations: an incorrect field affects eligibility or binding, and failure of a prerequisite then constrains later obligations. This remains a candidate explanation. No intervention here isolates the contributions of extraction, prompts, controller, validation, and model. The structure shared by B_ledger and A2 does not itself explain the difference between them. Moreover, B_ledger has more selection-validation failures, 24 versus 15, but a higher F1 score. Raw failure counts alone do not explain the ranking.

7. Limitations, Practical Relevance, and Author Contribution

The evaluation covers one DeepSeek V3.1 configuration and one run per method-trajectory pair. Four families with three variants each limit the independence of observations. Scenarios were exposed during development, and one was used in smoke tests. These conditions are not a held-out test set. Fixed seed and temperature values neither replace repetitions nor guarantee reproducible responses from a remote service.

Computation is not matched across conditions. A0 and the structured methods differ in prompts, numbers of stages, and output limits; B_ledger and A2 receive different histories following their queries. The A2/A0 comparison therefore concerns whole systems, while A2/B_ledger concerns two policies within a shared framework. It does not isolate a universal causal effect of “added memory.”

Semantic annotations were prepared with AI assistance, without an independent annotator or measurement of inter-rater agreement. Individual descriptions permit alternative explanations. Costs are confirmed by API responses, not by an independent invoice. The provider's implementation of all requested parameters was not directly verified. Complete logs support replaying the evaluation but do not promise identical new generations.

The evaluated simulator environment was English-language. This experiment does not establish multilingual performance. Real-world background monitoring, external side effects, time zones, and user utility were not evaluated. PM-Bench itself removes completed menu items, which also prevents an independent measurement of protection against repeated external actions.

Potential applications include evaluating assistants that track schedules, revise obligations, and respond when materials become available. The organisation of the logs helps distinguish omissions, incorrect conditions, wrong actions, and unsuccessful execution. Improved educational outcomes, user time savings, and deployment reliability remain untested possibilities.

The author's contribution includes framing the prospective-memory question and defining a controlled comparison, as documented in the study protocol and successive task specifications. Programming, log analysis, and manuscript preparation used AI tools, including Codex. Independent authorship of every line of code and independent human annotation are not claimed. PM-Bench and its evaluator belong to their original authors; architectural ideas are attributed to PIS and related work. The validity of the findings must be assessed against the protocol, recorded evaluations, and stated limitations; the existence of a repository alone is insufficient.

8. Conclusion

This project investigated prospective memory in language-model AI agents: executing a current intention at the appropriate time or event while respecting revisions, dependencies, and execution outcomes. The work progressed from instruction P1 to a structured store and programmatic monitoring. The A0/A1 pilots showed no improvement; a separate successful smoke test demonstrated functionality on a known scenario.

In the frozen comparison of 12 trajectories from four families, mean Set-F1 was 0.8643 for A0, 0.8111 for B_ledger, and 0.4806 for A2. The primary difference, A2 − B_ledger, was −0.330556; the secondary difference, A2 − A0, was −0.383730. A0 had the lowest API-reported cost. The improvement hypothesis was not supported in this comparison. The result is limited to one model configuration, scenarios exposed during development, and an absence of repeated runs; it does not establish that structured memory is generally ineffective.

Saved records show failures in intention revision, binding, and selection, including after the required evidence was obtained. Semantic error propagation remains a possible explanation, rather than an established cause of the entire deficit. Cancellations and dependencies must be accounted for alongside Set-F1, because not every unfinished obligation becomes an official omission.

One next diagnostic experiment could replace model-produced extraction and revision with reference annotations based only on information revealed so far, while preserving downstream system behaviour. Such annotations must not include future events, hidden answers, or completions that did not occur. This experiment has not been performed. It could help localise the role of record formation, but would not be a deployable system, an automatic performance upper bound, or proof of a single bottleneck.

9. References

External research literature

[1] Liu G., Gabriel S. PM-Bench: Evaluating Prospective Memory in LLM Agents. arXiv:2607.12385v1, 14 July 2026. URL: https://arxiv.org/abs/2607.12385v1.

[2] Rendell P. G., Craik F. I. M. Virtual week and actual week: Age-related differences in prospective memory. Applied Cognitive Psychology. 2000. Vol. 14, No. 7. pp. S43–S62. DOI: 10.1002/acp.770. URL: https://onlinelibrary.wiley.com/doi/10.1002/acp.770.

[3] Zhang T., Wang X., Zhang Q., Chen Q., Li K., Chen Y., Wang D., Meng H., Lu Y. TriggerBench: Investigating Prospective Memory for Large Language Models. arXiv:2606.23459v1, 22 June 2026. URL: https://arxiv.org/abs/2606.23459v1.

[4] Hu Y., Wang Y., McAuley J. Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions. arXiv:2507.05257. First released in 2025; version used: v4, 28 June 2026. URL: https://arxiv.org/abs/2507.05257v4.

[5] Zhao J., Wu C. Making Prospective Memory SLM-Shaped: Typed Intention Stores for Small-Model Agents. arXiv:2609.01272v1, 1 September 2026. URL: https://arxiv.org/abs/2609.01272v1.

Internal Kiodai project materials

[6] Kiodai. Frozen protocol and implementation, v2.1-comparison-v1. 10–11 September 2026. research/v2_1/comparison_v1.json; configs/v21_comparison_v1.json; kiodai_v2/agent.py, store.py; prompts/v2_1/. Versions are specified in Section 4.1.

[7] Kiodai. Primary records and final scores, v2.1-comparison-v1. 11 September 2026. results/v2_1/comparison-v1/: scenario.json, calls.jsonl, actions.jsonl, steps.jsonl, and score.json in 36 directories; comparison_report_reviewed.json. Inventory: research/v2_1/live_comparison_v1_inventory.json.

[8] Kiodai. Historical manuscript; RESULTS.json, FOLLOWUP_RESULTS.md, and FOLLOWUP_RESULTS.json; records live-pair-42433e00474b and live-pair-bdf241965f35. A0/A1 pilots, 8–9 September 2026.

[9] Kiodai. v2 development and separate v2.1 smoke test. docs/v2/DEEPSEEK_SMOKE_RESULTS.md; docs/v2_1/DEEPSEEK_SMOKE_RESULTS.md; results/v2_1/deepseek-smoke-v1/analysis.json. 9 September 2026.

[10] Kiodai. Post-run failure review. results/v2_1/comparison-v1/semantic_review.json, baseline_behavior_review.json; docs/v2_1/COMPARISON_RESULTS.md. 11 September 2026. AI annotations; key cases checked against raw logs.

External sources and local records verified on 12 September 2026. Full pointers, hashes, figure data, and recalculation results are retained in the existing verification materials. English edition prepared on 22 September 2026.